

Jira Assign AI

Production Changes Report

This document describes all architectural and algorithmic changes applied to the Jira Assign AI system as part of its transition from the Minimum Viable Product (MVP) stage to a production-ready deployment.

Project: Jira Assign AI — Intelligent Task Routing System
Author: Edip Zahit Güney
Date: March 2026

Contents

1	Executive Summary	4
1.1	The Problem	4
1.2	The Solution	4
1.3	Competitive Positioning	4
1.4	Production Upgrade Overview	4
2	Technical Stack & Architecture	5
2.1	Backend — Python / FastAPI	5
2.2	Frontend — React / Node / TypeScript	5
2.3	Database — PostgreSQL + pgvector	6
2.4	Nginx Reverse Proxy	6
2.5	Docker Containerization	6
3	Security, Privacy & Enterprise Readiness	6
3.1	JWT Authentication	6
3.2	Multi-Project Isolation	6
3.3	Jira Server & Data Center (On-Premise) Native	6
3.4	Absolute Data Sovereignty	6
3.5	Cryptographic Credential Management	7
4	The Core Algorithm: The 5-Factor Scoring Engine	7
4.1	Phase 1: Hard Constraints (Short-Circuit)	7
4.2	Phase 2: Soft Optimization — The Five Criteria	7
4.3	Strategy Presets	7
5	Advanced Scoring: Time-Decay Filtering	8
5.1	The Decay Formula	8
5.2	Affected Scoring Factors	8
6	Performance & Sync Strategy	9
6.1	Incremental Upsert	9
6.2	Three-Tier Sync Schedule	9
6.3	Automated Scheduler	9
6.4	Recommendation Engine — Workload Caching	9
6.4.1	The Problem	9
6.4.2	Optimization 1: Slim Payload Fetching	10
6.4.3	Optimization 2: In-Memory Workload Cache	10
6.4.4	Cache Invalidation	10
6.5	Frontend Ticket Caching (10-Minute TTL)	11
6.5.1	The Problem	11
6.5.2	The Solution	11
7	Database Schema (PostgreSQL)	11
7.1	Relationship Design & Integrity	12

8	Backend Logic Improvements	12
8.1	Incremental Sync	12
8.1.1	The Problem	12
8.1.2	The Solution	13
8.1.3	Implementation Details	13
8.1.4	Frontend Changes	14
8.2	Time-Decay Filtering	14
8.2.1	The Problem	14
8.2.2	The Decay Formula	14
8.2.3	Affected Scoring Factors	15
9	Containerization & Deployment	15
9.1	Docker & Nginx	15
9.1.1	Architecture	15
9.1.2	Nginx Configuration	15
9.1.3	Deployment Commands	16
9.2	Automated Nightly Sync & Manual Scheduler Toggle	16
10	Enterprise-Scale Architecture	16
10.1	PostgreSQL + pgvector Migration	16
10.1.1	Why SQLite Breaks at Scale	16
10.2	Three-Tier Sync Schedule	17
11	Multi-Project & Authentication	17
11.1	JWT Authentication	17
11.1.1	Account Seeding	17
11.1.2	Authentication Flow	18
11.2	Multi-Project Support	18
11.2.1	Database Schema Changes	18
11.2.2	Existing Data Migration	19
11.2.3	API Changes	19
11.2.4	Frontend Changes	19
11.3	Project Creation Wizard	19
11.4	Scoring Engine Refinements	20
11.4.1	Criterion 5 — Recent Activity: Reframed as Presence Consistency	20
11.4.2	Criterion 6 — Urgency: Removed	21
11.4.3	Dynamic Priority Mapping	21
11.4.4	Success Rate — Weight Reduced	21
12	Batch Migration Script	22
12.0.1	Why a Separate Script Is Necessary	22
12.0.2	Script Design	22
13	Security Hardening & Production Safeguards	22
13.1	HttpOnly Cookie Authentication	23
13.2	Enforced Environment Secrets	23
13.3	Brute-Force Protection	23
13.4	Nginx Security Headers	24
13.5	Admin Recovery CLI	24

13.6 Encryption Key Sovereignty	24
14 Conclusion	24
A MVP vs. Production Comparison	25
A.1 Detailed Scoring Criteria Comparison	26
B Frontend Architecture & Page Descriptions	26
B.1 Login Page	26
B.2 Dashboard	27
B.3 Projects Page	28
B.4 Weights Editor	29
B.5 Team Constraints	29
B.6 Project Setup Wizard	30
B.7 Project Context & Global State	30

Executive Summary

The Problem

Manual task assignment in Jira is slow, biased, and ignores developer capacity or hidden expertise. Project managers waste hours reading ticket descriptions to determine who should handle each issue, leading to uneven workload distribution and suboptimal use of team knowledge.

The Solution

Jira Assign AI is an intelligent, context-aware task routing system that ranks every team member from 0 to 100 for any given issue. Rather than relying on round-robin rotation or rigid *If-This-Then-That* rules, the system combines a rules engine with an Explainable AI (XAI) scoring layer — producing ranked recommendations that reflect each developer's actual expertise, current workload, reliability history, and recent activity.

Competitive Positioning

Existing tools typically offer simple round-robin rotation or static label-based routing. Cloud-based webhook solutions introduce data privacy concerns that are unacceptable in corporate environments. Jira Assign AI's hybrid approach — hard constraint filtering followed by a five-factor soft optimization engine — operates entirely on-premise with no data leaving the corporate network, while providing significantly more nuanced recommendations than any rule-based alternative.

Production Upgrade Overview

The Jira Assign AI system was initially developed as a fully functional MVP capable of demonstrating intelligent, NLP-powered task assignment on a single developer workstation. While the core algorithm, scoring engine, and user interface were complete and operational, several architectural limitations prevented the system from being deployed on a shared corporate intranet or scaled to enterprise-grade Jira installations.

This document details the production improvements organized into six areas:

1. **Backend Logic Improvements** — Incremental synchronization and time-decay scoring
2. **Containerization & Deployment** — Docker, Nginx, and automated sync scheduling
3. **Enterprise-Scale Architecture** — PostgreSQL migration and tiered sync
4. **Multi-Project & Authentication** — JWT auth, multi-project support, project creation wizard, and scoring engine refinements
5. **Batch Migration Script** — One-time standalone script for large historical dataset import
6. **Security Hardening & Production Safeguards** — HttpOnly cookies, enforced secrets, Nginx security headers, and admin recovery tooling

Table 1: Production Changes Overview

Change	Description
Incremental Sync	Upsert strategy replacing full wipe-and-refetch
Time-Decay Filtering	Exponential decay weighting on scoring engine
Docker & Nginx	Full stack containerization and reverse proxy
Automated Scheduler	Nightly sync + manual toggle in Settings UI
PostgreSQL + pgvector	Enterprise database with native vector search
Three-Tier Sync Schedule	Daily, weekly, and monthly sync tiers
JWT Authentication	Login system with bcrypt and bearer tokens
Multi-Project Support	Per-project isolation of all data and config
Project Creation Wizard	Guided multi-step setup for new projects
Scoring Engine Refinements	Urgency removed, Activity reframed, dynamic priority mapping
Batch Migration Script	Standalone tool for initial large-scale data import
Frontend Ticket Cache	10-minute TTL client-side persistence for responsiveness
Workload Cache	In-memory server-side cache with slim payload fetching
Brute-Force Protection	5 tries/minute rate limiting on login endpoint
Security Hardening	HttpOnly cookies, enforced secrets, Nginx headers, admin CLI

Technical Stack & Architecture

Backend — Python / FastAPI

Python is the natural choice for the backend due to the heavy mathematical workload at the core of the scoring engine. Cosine similarity matrix computation and the NLP library ecosystem — particularly `sentence-transformers` and `pgvector` — require Python's scientific computing stack. FastAPI provides asynchronous request handling with automatic OpenAPI documentation generation.

Frontend — React / Node / TypeScript

The React/TypeScript frontend creates a reactive, interactive dashboard capable of rendering the AI's complex output data — including radar charts for per-developer scoring breakdowns, real-time sync progress indicators, and multi-project navigation. TypeScript enforces type safety across the API contract boundary.

Database — PostgreSQL + pgvector

PostgreSQL is used as the production database for enterprise-grade reliability, concurrent access, and transactional integrity. The **pgvector** extension adds a native **vector** column type and HNSW indexing, enabling database-level cosine similarity search. This bypasses the RAM limitations and concurrency issues of local SQLite and eliminates the need to load embedding matrices into application memory.

Nginx Reverse Proxy

Nginx acts as the single entry point to the system. It serves the compiled React SPA as static files, proxies all `/api/` requests to the FastAPI backend, and provides an additional security layer at the network boundary.

Docker Containerization

Both the frontend and backend are fully containerized and managed by Docker Compose, enabling repeatable, stable deployments in corporate environments with a single **docker compose up** command.

Security, Privacy & Enterprise Readiness

The Jira Assign AI is architected specifically for enterprise environments where data privacy, corporate compliance, and network security are non-negotiable.

JWT Authentication

Administrative access to the application is protected by JWT-based authentication. Users must log in via a secure portal, and an admin user is seeded via environment variables at startup. Bcrypt is used for secure password hashing.

Multi-Project Isolation

All data — including credentials, task caches, team rules, and user assignments — is scoped by a **project_key**. This allows a single deployment to manage multiple distinct Jira projects with complete data isolation.

Jira Server & Data Center (On-Premise) Native

Designed to operate entirely within a locked-down corporate perimeter. No data leaves the network.

Absolute Data Sovereignty

All NLP inference (**all-MiniLM-L6-v2**) occurs locally on the server. No ticket data ever touches third-party AI APIs such as OpenAI.

Cryptographic Credential Management

Jira API tokens and PATs are encrypted using `cryptography.fernet` before being stored in the database.

The Core Algorithm: The 5-Factor Scoring Engine

The scoring engine evaluates users against new issues through a hybrid two-phase approach.

Phase 1: Hard Constraints (Short-Circuit)

Filters users based on project-specific Team Rules. For example, a ticket labelled `backend` will exclude all users not assigned to the Backend Team before any soft scoring begins.

Phase 2: Soft Optimization — The Five Criteria

1. **Expertise Score (Semantic Similarity):** Average cosine similarity of the top-5 closest historical tickets in the developer's resolved issue history, weighted by time-decay.
2. **Capacity Score (Workload):** Penalizes users with high task counts or story point totals relative to the team average.
3. **Reliability Score (Success Rate):** Based on on-time resolution rate and low reopen rate, decay-weighted to prioritize recent performance.
4. **Domain Score (Category Experience):** Counts label matches against the developer's historical tickets, with each match contributing its decay factor rather than a flat count.
5. **Momentum Score (Recent Activity):** Measures presence consistency over the last 10 working days — rewards regular contribution over sporadic bursts.

Note: Criterion 6 (Urgency/Velocity) from the MVP was removed to improve scoring accuracy. Urgency is a property of the task, not the developer — every candidate received an identical urgency score, contributing zero differentiation to the ranking. Ticket priority is still displayed in the UI as context but no longer participates in the scoring calculation. See Section [11.4](#) for full details.

Strategy Presets

Each project selects a strategy preset that distributes weight across the five criteria:

Table 2: Default Strategy Preset Weights

Criterion	BALANCED	SPEED	QUALITY	GROWTH
Expertise	35%	10%	45%	5%
Workload	30%	45%	10%	45%
Success Rate	5%	5%	15%	0%
Category Exp.	15%	5%	25%	5%
Recent Activity	15%	35%	5%	45%

Advanced Scoring: Time-Decay Filtering

In production, a developer's expertise from 2017 is often far less relevant than their work from 2024. The scoring engine applies an exponential time-decay multiplier to historical issue scores so that recent mastery dominates over stale expertise.

The Decay Formula

$$Score = Base_Similarity \times e^{-\lambda \cdot \Delta t}$$

Where Δt is the age of the resolved issue in years and λ controls the rate of decay. A floor of 0.30 is applied to ensure that deep historical expertise in a specialized domain is never completely zeroed out — a developer with many old tickets in a niche area still retains 30% of that signal.

```
@staticmethod
def _calculate_decay_factor(resolved_at) -> float:
    age_years = (today - resolved_date).days / 365.0
    decay = math.exp(-LAMBDA * age_years)
    return max(0.3, decay)
```

Listing 1: Time-decay formula — scoring_engine.py

Table 3: Decay Factor by Issue Age

Issue Age	Decay Factor	Effect
Today	1.00	Full weight
1 year ago	0.85	Slight reduction
2 years ago	0.70	Moderate reduction
3 years ago	0.55	Significant reduction
5+ years ago	0.30	Floor (preserves domain signal)

Affected Scoring Factors

The decay multiplier is applied to three of the five criteria:

1. **Expertise Score:** Both Jaccard tag similarity and NLP semantic similarity are weighted by each historical task's decay factor using a custom `_decay_weighted_top_k_similarity` method.

2. **Category Experience:** Each matching task contributes its decay factor instead of a flat count — five recent matches score higher than seven old matches.
3. **Success Rate:** On-time and quality rates are decay-weighted so recent performance dominates over historical failures.

Performance & Sync Strategy

To handle hundreds of thousands of tickets efficiently, the system uses a multi-tier sync engine designed to minimize API calls, NLP re-computation, and database downtime.

Incremental Upsert

Unlike the MVP's wipe-and-refetch approach, the production system only fetches issues updated since the last sync date. New issues are inserted; existing issues are updated in-place. The database is never empty during a sync window.

Three-Tier Sync Schedule

Table 4: Three-Tier Sync Schedule

Tier	Frequency	Scope
Nightly	Every night 02:00	Incremental: issues updated in last 24 hours
Weekly	Every Sunday 03:00	Partial: last 3 months — catches retroactive edits and relabeling
Monthly	1st of each month	Full resync: entire history — catches deletions and reorganizations

Automated Scheduler

An integrated background scheduler (via APScheduler) handles the three sync tiers automatically based on per-project configuration. A manual toggle in the Settings UI allows administrators to pause or resume the scheduler without modifying server configuration.

Recommendation Engine — Workload Caching

To ensure recommendation performance remains stable at enterprise scale (30,000+ open tickets), a dedicated in-memory workload cache was implemented in the recommendation engine.

The Problem

Each call to `/api/recommendations` previously required the backend to fetch every open assigned issue across the entire Jira instance in order to compute each developer's current workload. For a project with 1,200 open issues, this fetch alone took approximately 14 seconds. Projected to 30,000 issues, the same operation would have reached 3+ minutes per recommendation request — rendering the feature unusable at scale.

Optimization 1: Slim Payload Fetching

The root cause of the latency was the payload size per issue. The original fetch requested 16+ fields per ticket (summary, description, worklogs, comments, etc.), most of which are irrelevant to a workload calculation. The workload scoring criterion requires only two fields: `assignee` and `story_points`.

A targeted API call was implemented that requests only these two fields:

- **Payload before:** ≈ 100 KB per issue
- **Payload after:** ≈ 200 bytes per issue
- **Reduction factor:** $\approx 500\times$ smaller per issue

Optimization 2: In-Memory Workload Cache

Even with a slim payload, fetching all open tickets on every recommendation request wastes bandwidth and Jira API quota. A thread-safe singleton cache was implemented in `backend/workload_cache.py` to store the parsed workload map across requests.

```
# Data structure: Dict[user_id, List[Dict]]
# Maps each developer to their list of open task summaries.
# Thread-safe singleton with a 10-minute TTL.
workload_cache = WorkloadCache(ttl_seconds=600)
```

Listing 2: Workload cache — `workload_cache.py`

Cache behaviour:

- **Cache Miss (first request):** Backend performs the slim-payload Jira fetch, parses the result, and stores the workload map. Response time: 1–3 seconds (down from 14 seconds with the original full-payload fetch).
- **Cache Hit (subsequent requests):** Workload map is returned directly from memory. Response time: < 0.5 seconds.
- **Memory footprint:** ≈ 1.9 MB for 30,000 open issues.

Table 5: Recommendation Engine Performance — Before vs. After

Scenario	Before	After (Cache Miss)	After (Cache Hit)
1,200 open issues	~ 14 s	$\sim 1-3$ s	< 0.5 s
30,000 open issues	> 3 min	$\sim 1-3$ s	< 0.5 s

Cache Invalidation

The cache is invalidated in two scenarios to ensure accuracy:

1. **TTL Expiry:** The cache self-expires after 10 minutes. The next request triggers a fresh slim-payload fetch automatically.
2. **Manual Sync:** Any user-triggered sync (Incremental or Full Re-Sync) immediately wipes the workload cache, ensuring that the next recommendation reflects the newly updated database state rather than stale pre-sync data.

Frontend Ticket Caching (10-Minute TTL)

To further reduce unnecessary Jira API load and improve the responsiveness of the Dashboard, a client-side caching layer was implemented for fetched tickets.

The Problem

In the MVP, every time the user navigated to the Dashboard or switched projects, a fresh network request was made to the `/api/unassigned` endpoint. Even if the data had been fetched seconds prior, the application would show a loading state and re-fetch the entire ticket list, causing unnecessary delay and server load during frequent project switching.

The Solution

The frontend now implements a **10-minute Time-To-Live (TTL)** cache using `localStorage`.

- **Project-Scoped:** Each project's ticket list is cached independently using a key format: `issues_cache_{project_key}`.
- **Persistence:** Because it uses `localStorage`, the cache persists across page refreshes and browser restarts.
- **Automatic Invalidation:** Each cache entry includes a timestamp. If a request is made and the cached data is older than 10 minutes, the application automatically triggers a fresh background fetch and updates the cache.
- **Manual Override:** Any manual sync trigger (Incremental or Full) immediately invalidates the frontend cache to ensure the user sees the latest results from the backend.

Database Schema (PostgreSQL)

The system uses a normalized relational schema optimized for enterprise scale, multi-project isolation, and high-performance vector search.

- **jira_users** — Global identity registry for Jira user accounts to ensure consistent identification across multiple projects.
- **portal_users** — Local administrator accounts for the portal. Stores hashed passwords (bcrypt) and provides the foundation for JWT session management.
- **projects** — The central multi-project registry. Stores Jira credentials (Fernet-encrypted), synchronization state, active strategy preset, and per-project sync schedules.
- **task_cache** — The primary data store for historical issues. Contains metadata, reliability metrics (`completed_on_time`, `reopen_count`), and the `vector(384)` embeddings used for similarity matching.
- **developer_activity_cache** — Caches recent worklog patterns and leave dates in JSON format to accelerate the "Recent Activity" scoring calculations and leave detection.

- **user_teams** — Maps Jira users to specific project teams. Supports a **GLOBAL** project key for users who participate across the entire organization.
- **team_rules** — Stores label-to-team hard constraints per project, used to filter candidates in Phase 1 before soft scoring begins.
- **advanced_settings** — Stores project-specific engine constants, including granular weight overrides and scoring saturation thresholds.
- **settings** — A global key-value store for application-wide configurations that do not belong to a specific project.

Relationship Design & Integrity

All tables referencing project data carry a **project_key** foreign key with **CASCADE DELETE** enabled. This ensures absolute data isolation; removing a project automatically and cleanly purges all its associated historical tasks, team assignments, rules, and vector embeddings from the system without orphan records.

Backend Logic Improvements

This section covers algorithmic improvements that require no infrastructure changes. These modifications enhance the accuracy and efficiency of the system's core data pipeline and scoring engine.

Incremental Sync

The `/api/sync` endpoint was converted from a destructive full-wipe strategy to an incremental upsert, with a preserved full re-sync option for periodic maintenance.

The Problem

The MVP's synchronization endpoint deleted the entire **task_cache** table on every call and re-fetched all historical issues from Jira. For a project with 5,000+ resolved issues, this meant:

- Every sync required re-downloading thousands of issues via paginated API calls
- Every embedding vector had to be re-computed by the NLP model — expensive CPU time
- The database was empty during the sync window, rendering the system unusable
- Jira API rate limits were hit frequently on large projects

The Solution

The sync endpoint now operates in two modes:

1. **Incremental Sync (Default):** Reads the `last_sync_date` from the `Setting` table, constructs a JQL query `updated >= "YYYY-MM-DD"`, and fetches only issues modified since the last sync. New issues are inserted; existing issues are updated in-place (upsert).
2. **Full Re-Sync (?full=true):** Preserves the original wipe-and-refetch behavior for periodic maintenance, recommended weekly or monthly.

Implementation Details

New JQL Method:

```
def get_issues_updated_since(self, since_date: str):
    jql = (
        f'project = {self.project_key} '
        f'AND updated >= "{since_date}" '
        f'ORDER BY updated DESC'
    )
    # Fetches only recently changed issues via pagination
```

Listing 3: Incremental JQL Filter — `jira_client.py`

Upsert Strategy:

For each fetched issue, the system queries the database by `issue_key`. If the record exists, it is updated in-place; if not, a new record is inserted. This means:

- Unchanged issues are never touched — no I/O or NLP re-computation
- Modified issues have their fields and embeddings refreshed
- New issues are appended to the cache
- The database is **never empty** during sync — always queryable

Cold-Start Safety: If no `last_sync_date` exists (first-ever sync), the endpoint automatically falls back to a full sync.

Enhanced API Response:

```
{
  "sync_mode": "incremental",
  "fetched_from_jira": 42,
  "inserted": 15,
  "updated": 27,
  "skipped": 0,
  "time_seconds": 3.2,
  "last_sync_date": "2026-03-25"
}
```

Listing 4: Enriched sync response payload

Frontend Changes

The Settings page now displays two sync actions:

- A blue primary button labeled “**Incremental Sync**” for daily use
- An amber outlined secondary button labeled “**Full Re-Sync**” for periodic maintenance

Each button triggers a mode-specific confirmation modal. After completion, the UI shows detailed statistics including mode, new/updated counts, and duration.

Time-Decay Filtering

A time-decay multiplier was added to the scoring engine so that recent work outweighs older historical data when calculating developer expertise and reliability.

The Problem

The MVP scoring engine treated all historical tasks equally. A developer who solved a WebSocket performance issue 7 years ago received the same expertise credit as a developer who solved the same type of issue last month. In rapidly evolving codebases, this leads to stale recommendations.

The Decay Formula

The engine applies exponential decay using the formula:

$$Score = Base_Similarity \times e^{-\lambda \cdot \Delta t}$$

Where Δt is the age of the issue in years and λ is the decay constant. A floor of 0.30 ensures deep historical expertise is never entirely discarded.

```
@staticmethod
def _calculate_decay_factor(resolved_at) -> float:
    age_years = (today - resolved_date).days / 365.0
    decay = math.exp(-LAMBDA * age_years)
    return max(0.3, decay)
```

Listing 5: Time-decay formula — scoring_engine.py

Table 6: Decay Factor by Issue Age

Issue Age	Decay Factor	Effect
Today	1.00	Full weight
1 year ago	0.85	Slight reduction
2 years ago	0.70	Moderate reduction
3 years ago	0.55	Significant reduction
5+ years ago	0.30	Floor (preserves domain signal)

The floor of 0.30 ensures that deep historical expertise is never completely zeroed out — a developer with many old tickets in a specialized domain still retains 30% of that signal.

Affected Scoring Factors

The decay multiplier was applied to three criteria:

1. **Expertise Score:** Both Jaccard tag similarity and NLP semantic similarity are weighted by each historical task's decay factor using a custom `_decay_weighted_top_k_similarity` method.
2. **Category Experience:** Each matching task contributes its decay factor instead of a flat count — 5 recent matches score higher than 7 old matches.
3. **Success Rate:** On-time and quality rates are decay-weighted so recent performance dominates over historical failures.

Containerization & Deployment

This section covers the transition from a local development setup to a containerized, production-ready deployment startable with a single command on any server.

Docker & Nginx

The full stack was containerized behind an Nginx reverse proxy using Docker Compose.

Architecture

1. **Backend Container** (`jira-ai-backend`) — `python:3.11-slim` base, multi-stage build with the `all-MiniLM-L6-v2` NLP model baked in at build time. No internet required at runtime. Runs `uvicorn main:app` on internal port 8000.
2. **Frontend Container** (`jira-ai-frontend`) — Multi-stage build: `node:20-alpine` compiles the React SPA, `nginx:alpine` serves the output and acts as reverse proxy on host port 80.

Nginx Configuration

```
server {
    listen 80;
    server_name localhost;

    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://backend:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_read_timeout 300s;
    }
}
```



```
    proxy_connect_timeout 30s;
    proxy_send_timeout 300s;
  }
}
```

Listing 6: nginx.conf — Reverse proxy configuration

Deployment Commands

```
docker compose build      # ~5 minutes on first run
docker compose up -d      # start in detached mode
docker compose logs -f    # view logs
docker compose down       # stop
```

Listing 7: Production deployment workflow

Automated Nightly Sync & Manual Scheduler Toggle

The `/api/sync` endpoint was configured to run automatically on a nightly schedule, eliminating manual sync triggers. A manual scheduler toggle was also added to the Settings UI — a toggle button with a confirmation modal and persistent state in the database — allowing the administrator to pause or resume scheduled sync without modifying server configuration.

Nightly incremental sync runs at 02:00 AM server time. **Weekly full re-sync** runs every Sunday at 03:00 AM. Within the Docker deployment, a lightweight scheduler container handles this at the infrastructure level using `mcuadros/ofelia`.

```
0 2 * * * curl -s -X POST http://localhost:8000/api/sync
0 3 * * 0 curl -s -X POST http://localhost:8000/api/sync?full=
true
```

Listing 8: Linux cron entries

Enterprise-Scale Architecture

Context: A large enterprise Jira installation — for example, 7+ years of data on Jira Premium Server at a defense or aerospace company — presents challenges fundamentally different from a typical team project.

PostgreSQL + pgvector Migration

SQLite was replaced with PostgreSQL extended by the `pgvector` plugin, enabling native, indexed vector similarity search inside the database engine.

Why SQLite Breaks at Scale

- **Concurrency:** File-level locking causes concurrent API requests to queue and serialize.

- **Memory:** Loading 200,000+ 384-dimensional vectors into RAM via NumPy consumes 2–3 GB per request.
- **No Native Vector Search:** Every similarity query is a full table scan. PostgreSQL’s pgvector provides HNSW indexes reducing search from $O(n)$ to $O(\log n)$.

```
SELECT issue_key, assignee,
       1 - (embedding <=> $1) AS cosine_similarity
FROM task_cache
WHERE assignee = $2
ORDER BY embedding <=> $1
LIMIT 5;
```

Listing 9: pgvector similarity query

Vectors never leave the database — the application only receives the top matches, eliminating RAM spikes entirely.

Three-Tier Sync Schedule

Table 7: Three-Tier Sync Schedule

Tier	Frequency	Scope
Nightly	Every night 02:00	Incremental: issues updated in last 24 hours
Weekly	Every Sunday 03:00	Partial: last 3 months — catches retroactive edits and relabeling
Monthly	1st of each month	Full resync: entire history — catches deletions and reorganizations

Multi-Project & Authentication

This section introduces user authentication, multi-project support, a guided onboarding flow, and significant refinements to the scoring engine based on real-world usage feedback.

JWT Authentication

JWT-based authentication was added to protect all application endpoints. The system uses a single shared account model — appropriate for a corporate intranet tool used by a small team of project managers. No open registration endpoint exists.

Account Seeding

The administrator account is created automatically on first startup from environment variables. If no users exist in the database, the backend reads `ADMIN_USERNAME` and `ADMIN_PASSWORD` from the environment, hashes the password using `bcrypt` via `passlib`, and inserts the account. Plain text credentials never touch the database. If the environment variables are missing on first startup, the application raises an error and refuses to start.

```
ADMIN_USERNAME=admin
ADMIN_PASSWORD=admin
JWT_SECRET=<64-char hex string generated via secrets.token_hex
(32)>
JWT_EXPIRE_HOURS=24
```

Listing 10: Required environment variables

Authentication Flow

1. User opens the application — no token found → redirected to `/login`
2. User submits credentials → `POST /auth/login` → backend verifies bcrypt hash → returns signed JWT token
3. Frontend stores token in an `HttpOnly` cookie; all subsequent API requests include `Authorization: Bearer <token>` header
4. On 401 response from any endpoint, frontend clears the token and redirects to `/login`
5. After 24 hours the token expires; user logs in again to receive a fresh token
6. Logout clears the token — no server-side blacklist required

Multi-Project Support

The system was extended to support multiple Jira projects concurrently. Every database entity is now scoped to a `project_key`, allowing each project to maintain its own independent team constraints, scoring configuration, sync history, and task cache.

Database Schema Changes

A new `Project` table was introduced as the root entity:

- `key` — Primary Key (e.g. `ASAI`)
- `name` — Display name
- `server_url`, `user_email`, `api_token` — Optional per-project credential overrides. If `null`, the system falls back to global credentials in the `Setting` table.
- `active_strategy`, `override_labels` — Per-project scoring configuration
- `last_sync_date` — ISO timestamp of most recent sync

The following tables were updated to include a `project_key` foreign key with cascade delete: `TaskCache` (indexed, NOT NULL), `TeamRule` (strictly project-scoped), `AdvancedSettings` (primary key changed to `project_key` — one settings row per project), and `UserTeam` (primary key changed to `(user_account_id, project_key)`, with `project_key = "GLOBAL"` indicating cross-project membership).

Existing Data Migration

On schema upgrade, all existing `TaskCache`, `TeamRule`, and `UserTeam` rows were tagged with the project key from the current `Setting` table, preserving all cached embeddings and configuration without data loss.

API Changes

- `GET /api/projects` — List all configured projects
- `POST /api/projects` — Create a new project
- `PUT /api/projects/{key}` — Update project name, strategy, labels, or credential overrides
- `DELETE /api/projects/{key}` — Remove project and all cascaded data
- All existing endpoints now require a `project_key` header validated against the `Project` table.

Frontend Changes

- **Sidebar:** Expandable project list showing each project's key badge and display name. Active project highlighted with a colored left border. Unsynced projects display an amber warning indicator.
- **Global State:** `activeProject` managed globally and persisted in `localStorage`. All API requests pass `activeProject` as a request header.
- **Dashboard:** Data re-fetches immediately on project change. If `last_sync_date` is `null`, a prominent warning overlay replaces the dashboard content.
- **Settings:** Reorganized into a Global Credentials section and a Projects section with per-project General and Credentials tabs.

Project Creation Wizard

A guided, multi-step wizard was implemented to streamline the creation and initial configuration of Jira projects. While subsequent project additions can be managed via the Settings interface, the wizard is the primary entry point for new installations and simplifies the complex handshake between Jira authentication, project selection, and initial data synchronization.

If the application detects that zero projects are configured, the user is automatically redirected to the wizard to complete the mandatory onboarding steps.

The wizard walks through four distinct phases:

1. **Jira Connection** — Validating the Jira server URL and credentials (Email + API Token) with real-time feedback.
2. **Live Project Selection** — Fetching a list of available projects from the Jira instance, allowing the administrator to select a project by name rather than manually entering a Jira key.

3. **Team & Constraint Definition** — Defining the project's internal teams and initial label-to-team routing rules before the first sync.
4. **Initial Synchronization** — Triggering the first high-volume fetch and NLP embedding generation, with a real-time progress bar ensuring the dashboard remains locked until the system is ready for use.

The wizard is implemented as a React modal component with persistent state tracking, allowing users to re-enter the flow from the Settings page if a setup was interrupted.

Scoring Engine Refinements

Following real-world usage and design review, three scoring criteria were revised and one was removed.

Criterion 5 — Recent Activity: Reframed as Presence Consistency

The original implementation summed total logged seconds over the last 30 days, which inadvertently rewarded developers who worked excessive hours — creating a contradiction with Criterion 2 (Workload), which penalizes high task load. A developer grinding long weeks would simultaneously be penalized for high workload and rewarded for high activity.

The criterion was reframed as a **presence consistency** metric. Instead of volume, it measures regularity. Additionally, the presence check window was extended from 4 to 7 days to better handle weekend gaps and prevent false "inactive" flags on Monday mornings.

```
active_days = count_distinct_days_with_any_worklog(  
    last_10_working_days)  
# 7 active days out of 10 = full score  
activity_score = min(100, (active_days / 7) * 100)
```

Listing 11: Revised Recent Activity formula

Key changes:

- **Observation Window:** Reduced from 30 days to the last 10 working days to emphasize current momentum.
- **Weekend Gap Protection:** The "current presence" check now uses a 7-day look-back window (up from 4) to ensure developers returning from a standard weekend are not incorrectly flagged as inactive.
- **Leave Detection Upgrade:** Transitioned from a "unique days" to a **cumulative hours** calculation for leave detection. This ensures that long-term leave (e.g., a single 120-hour entry covering 3 weeks) is correctly identified, ensuring "On Leave" warnings are accurate.
- **Holiday Handling:** If a developer has zero worklogs AND zero new task assignments in the window, their score is treated as neutral (50) rather than zero — absence may indicate planned leave, not ghosting.

Criterion 6 — Urgency: Removed

The urgency criterion was removed from the scoring formula entirely. The core problem was fundamental: urgency is a property of the *task*, not the *developer*. Every developer received the identical urgency score for a given ticket, meaning the criterion contributed zero differentiation between candidates. It was mathematically present in the formula but had no effect on the ranking order — it shifted every score by the same constant, leaving the relative ordering completely unchanged.

Removing it reduces the scoring engine from six criteria to five. The strategy preset weights were redistributed across the remaining criteria accordingly. Ticket priority is still read from Jira and displayed in the UI as context for the project manager, but it no longer participates in the scoring calculation.

Dynamic Priority Mapping

The previous hardcoded priority-to-score mapping (Highest→100, High→75, etc.) assumed standard Atlassian priority names. Projects using custom priority names (Blocker, Critical, Normal) would silently receive incorrect urgency signals.

The mapping is now fetched dynamically from Jira on each project sync:

```
priorities = fetch_priorities_ordered_by_rank(project_key)
scores = [100, 75, 50, 25, 10]
priority_score_map = {
    p.name: scores[min(i, len(scores)-1)]
    for i, p in enumerate(priorities)
}
```

Listing 12: Dynamic priority mapping

The mapping is stored in `AdvancedSettings` per project and can be manually overridden by the administrator in the Advanced Settings UI.

Success Rate — Weight Reduced

The Success Rate criterion was retained but its default weight was significantly reduced across all strategy presets. Deadline data in Jira is frequently unreliable — developers assigned to unrealistic deadlines consistently appear to underperform regardless of actual skill. Reducing the weight to 5–10% in most presets ensures this signal informs but does not dominate the recommendation.

Table 8: Updated Default Strategy Preset Weights

Criterion	BALANCED	SPEED	QUALITY	GROWTH
Expertise	35%	10%	45%	5%
Workload	30%	45%	10%	45%
Success Rate	5%	5%	15%	0%
Category Exp.	15%	5%	25%	5%
Recent Activity	15%	35%	5%	45%

Batch Migration Script

A standalone one-time migration script was developed for the initial import of large historical datasets spanning multiple years.

Why a Separate Script Is Necessary

The first-ever sync of 7+ years of Jira data cannot be handled by the regular `/api/sync` endpoint. At 200,000+ issues:

- Sequential NLP inference on CPU would take 12–48 hours inside an HTTP request that will time out
- A single network interruption requires restarting from zero with no recovery mechanism
- Jira API rate limits throttle the process unpredictably
- Loading the entire dataset into memory at once exceeds available RAM

Script Design

The migration script runs as a standalone Python process directly on the server, outside the web application:

1. **Resume Checkpoints:** Progress is saved after every batch of 100 tickets. On restart, the script continues from the last completed batch.
2. **Batch Encoding:** The NLP model's batch encoding mode delivers 10–20x faster inference than encoding one ticket at a time:

```
embeddings = model.encode(batch_texts, batch_size=32)
```
3. **Rate Limit Handling:** Automatic exponential backoff and retry on HTTP 429 responses. The script never crashes on rate limits.
4. **GPU Acceleration:** If a CUDA-capable GPU is available, the script uses `device='cuda'`, reducing multi-day migrations to hours.
5. **Progress Reporting:** Real-time console output shows batch number, elapsed time, estimated time remaining, and total issues processed.

Once the initial migration completes, the regular three-tier sync schedule takes over and the migration script is never needed again.

Security Hardening & Production Safeguards

To meet corporate compliance requirements and protect sensitive Jira integration data, a dedicated set of production-hardening measures was implemented across the authentication layer, reverse proxy configuration, and secret management strategy.

HttpOnly Cookie Authentication

Session token storage was migrated from `localStorage` to server-sent `HttpOnly` cookies. Tokens stored in `localStorage` are accessible to any JavaScript executing in the browser context — a significant attack surface if any third-party script or XSS vector is present. An `HttpOnly` cookie is transmitted automatically by the browser on every request but is never readable by JavaScript, completely eliminating the token-stealing XSS attack class.

Enforced Environment Secrets

All insecure default fallback values — including any hardcoded `super-secret-key` strings — were removed from the codebase. The application now performs a **safe crash** on startup if `JWT_SECRET` or `ENCRYPTION_KEY` are absent from the environment. Rather than silently falling back to a weak default, the backend raises a `ValueError` and refuses to start, making misconfigured deployments immediately visible rather than silently insecure.

```
JWT_SECRET = os.environ.get("JWT_SECRET")
ENCRYPTION_KEY = os.environ.get("ENCRYPTION_KEY")

if not JWT_SECRET or not ENCRYPTION_KEY:
    raise ValueError(
        "JWT_SECRET and ENCRYPTION_KEY must be set. "
        "Refusing to start with missing secrets."
    )
```

Listing 13: Enforced secret validation on startup

Brute-Force Protection

To protect the administrator account from automated password-guessing attacks, the login endpoint implements mandatory rate limiting.

- **Threshold:** Access is limited to **5 login attempts per minute** per IP address.
- **Implementation:** The system uses `slowapi` (a Python implementation of the Fixed Window rate limiting algorithm), integrated directly into the FastAPI routing layer.
- **Behavior:** If the threshold is exceeded, the backend returns an HTTP 429 Too Many Requests status code, and subsequent attempts are blocked until the one-minute window expires. This prevents brute-force scripts from cycling through thousands of common passwords at high speed.

```
@router.post("/login")
@limiter.limit("5/minute")
def login(request: Request, response: Response, ...):
    # Authenticate user...
```

Listing 14: Rate limiting on login endpoint

Nginx Security Headers

The Nginx reverse proxy configuration was extended to inject the following security headers on all responses:

- **X-Frame-Options: DENY** — Prevents the application from being embedded in an `<iframe>`, mitigating clickjacking attacks.
- **X-Content-Type-Options: nosniff** — Instructs browsers not to MIME-sniff responses away from the declared `Content-Type`, preventing content-type confusion attacks.
- **Strict-Transport-Security** — Enforces HTTPS connections (HSTS) for all future requests from compliant browsers, eliminating protocol downgrade attacks.

```
add_header X-Frame-Options "DENY" always;  
add_header X-Content-Type-Options "nosniff" always;  
add_header Strict-Transport-Security  
    "max-age=31536000; includeSubDomains" always;
```

Listing 15: Security headers in nginx.conf

Admin Recovery CLI

A `reset_password.py` utility was added as a standalone server-side script for manual administrator password resets. This eliminates the need for SMTP or email infrastructure — inappropriate for a locked-down internal tool — while providing a secure, auditable recovery path when administrator credentials are lost. The script accepts the new password via a prompt, hashes it with `bcrypt`, and updates the `portal_users` table directly. It is designed to be run exclusively on the host server by an operator with direct access to the deployment environment.

Encryption Key Sovereignty

Local `.encryption_key` files that previously stored Fernet cryptographic keys in the container's writable layer were eliminated. All cryptographic keys must now be supplied exclusively via secure environment variables. This ensures that no sensitive key material is ever persisted to disk inside the container, preventing key exposure through container image snapshots, accidental commits, or filesystem access.

Conclusion

The production improvements documented in this report transform Jira Assign AI from a local development prototype into a fully deployable, multi-project, authenticated system ready for corporate intranet use at enterprise scale.

The backend logic improvements ensure the scoring algorithm produces accurate, time-aware recommendations by prioritizing recent developer activity using an exponential decay model. Containerization packages the entire stack into portable Docker containers deployable with a single command. The enterprise-scale database upgrade replaces SQLite

with PostgreSQL and pgvector, eliminating RAM spikes and enabling millisecond-level vector search at any dataset size. The multi-project and authentication layer adds JWT-protected access, per-project configuration isolation, a guided project creation wizard, and significant scoring engine refinements — including the reframing of Recent Activity as a presence consistency metric, the removal of the flat Urgency criterion, and dynamic priority mapping for cross-project compatibility. The batch migration tooling handles the initial import of large historical datasets outside the web application’s request lifecycle. The frontend performance is further optimized by a persistent 10-minute ticket caching layer. The recommendation engine’s workload caching — combining slim payload fetching with an in-memory TTL cache — reduces workload fetch latency from over 14 seconds to under 0.5 seconds on repeated requests, ensuring the feature remains responsive at 30,000+ open tickets. Finally, the security hardening layer eliminates an entire class of XSS-based token theft, enforces the presence of cryptographic secrets at startup, implements brute-force rate-limiting on sensitive endpoints, injects industry-standard browser security headers, and provides an operator-grade password recovery mechanism — ensuring the system meets corporate compliance requirements from day one.

The core of the system — the five-factor scoring engine, the Explainable AI reasoning layer, the strategy presets (BALANCED, SPEED, QUALITY, GROWTH, PERSONALIZED), and the bilingual React dashboard — remains the foundation at every scale. Only the infrastructure, authentication layer, security posture, and project management capabilities grew with the requirements.

MVP vs. Production Comparison

The table below summarises the key capability differences between the original MVP prototype and the current production system.

Table 9: MVP vs. Production — Feature Comparison

Feature	MVP State	Production State
Database	SQLite	PostgreSQL + pgvector
Auth	None	JWT + Login System
Projects	Single	Multi-Project Selector
Syncing	Full re-fetch	Incremental + 3-Tier Scheduling
Teams	Global	Project-Isolated Teams
Deployment	Manual scripts	Docker + Nginx Compose
Weights	Static presets	Dynamic Weight & Engine Editor
Scoring	6 criteria	5 criteria + Time-Decay
Scaling	Limited (SQLite)	Enterprise-Ready (PostgreSQL)
Security	None	HttpOnly Cookies, HSTS, Enforced Secrets
Ticket Caching	None	10-Minute Frontend TTL
Workload Caching	None	In-Memory Cache + Slim Payload Fetch
Brute-Force Protection	None	5 Tries/Minute Rate Limit

Detailed Scoring Criteria Comparison

The following table provides a direct comparison of how specific scoring metrics evolved between the MVP and Production versions to improve accuracy and reduce bias.

Table 10: Scoring Criteria Evolution

Criterion	MVP (Legacy)	Production (Optimized)
Expertise	Pure semantic similarity; historical tasks weighted equally.	Semantic similarity + Exponential Time-Decay (recent tasks dominate).
Workload	Flat ticket count penalty; rewarded inactive users too heavily.	Normalized penalty relative to team average; dynamic scaling.
Reliability	Lifetime success rate; ancient failures penalize users forever.	Decay-weighted success rate; recent performance prioritized.
Category Exp.	Flat label match count; ignores skill evolution.	Decay-weighted label contribution; rewards recent focus.
Recent Activity	Total seconds logged (30d); rewarded excessive overtime.	Presence Consistency (distinct days in 10d window); rewards reliability.
Urgency	Priority-to-score mapping; applied identically to all users.	REMOVED ; Urgency is a task property, not a developer trait.

Frontend Architecture & Page Descriptions

The React/TypeScript frontend is a single-page application (SPA) compiled by Vite and served by Nginx. All views share a global `activeProject` context — switching projects triggers an immediate data re-fetch across every page. The application uses `localStorage` to persist the active project and relies on `HttpOnly` cookies for session management.

Login Page

The secure entry point for administrators. Presents a dark-themed credential form matching the application design system. On successful authentication the backend returns a signed JWT via an `HttpOnly` cookie; on failure an inline error is shown without revealing whether the username or password was incorrect. Unauthenticated navigation to any route redirects here automatically.

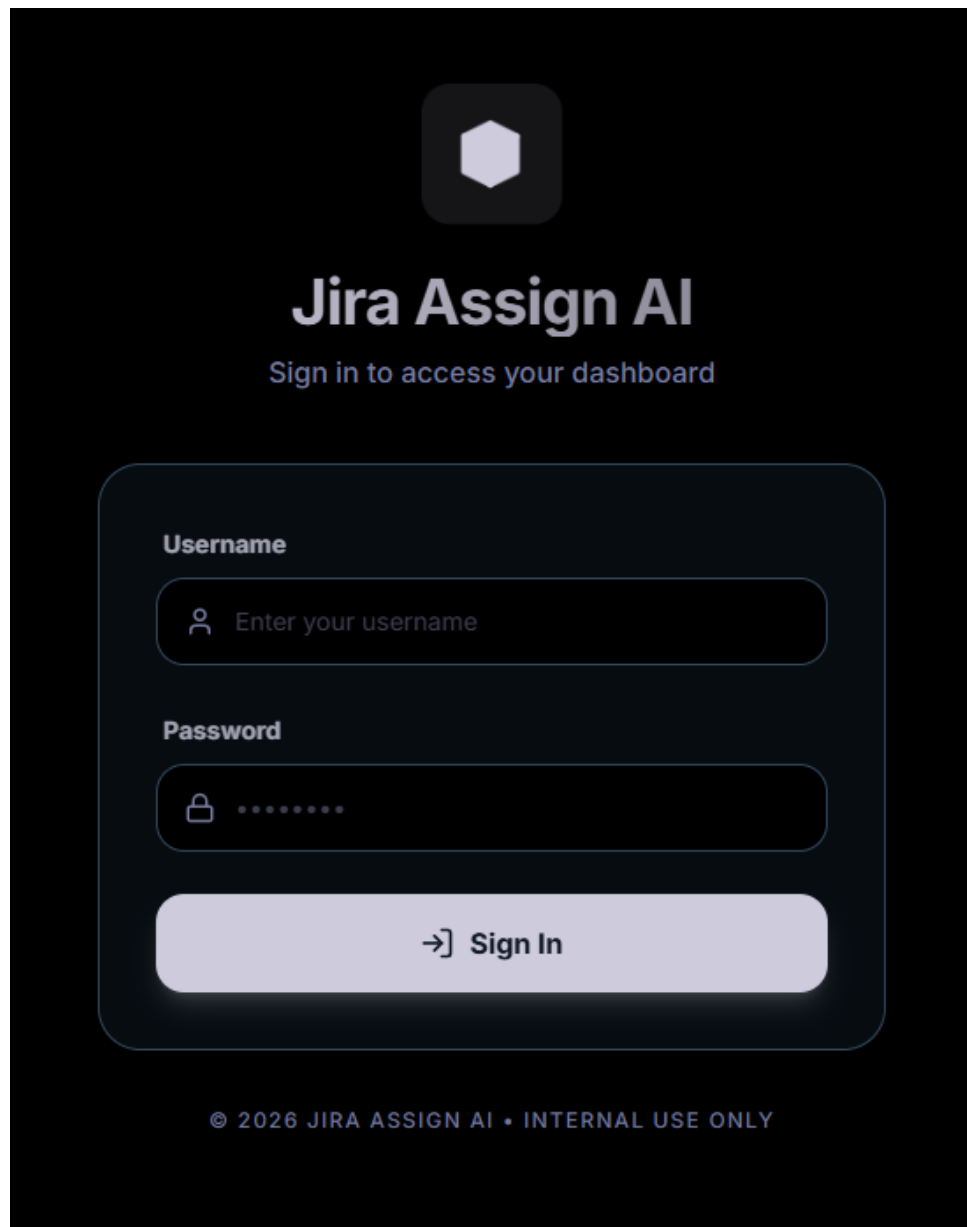


Figure 1: Login Page

Dashboard

The main workspace. Displays a ranked recommendation list for the currently selected Jira issue, with per-developer radar charts showing the breakdown across all five scoring criteria. If the active project has never been synced (`last_sync_date` is `null`), a warning overlay replaces the content and prompts the user to run an initial sync. Manual sync triggers are available here alongside the automated scheduler.

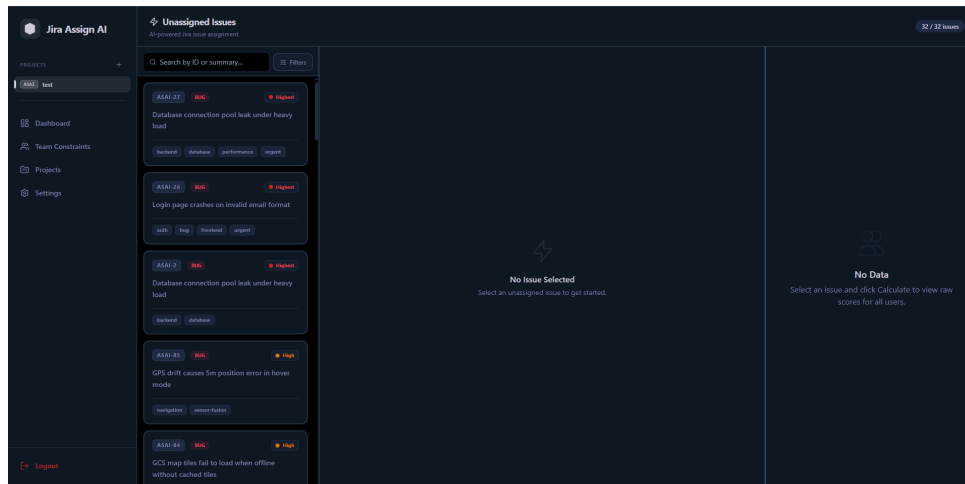


Figure 2: Dashboard

Projects Page

Central management for all configured Jira projects. Organised into two panels: the left panel lists all projects with their key badge, display name, last sync timestamp, and an amber warning indicator for unsynced projects; the right panel shows the selected project's General tab (strategy preset, label overrides) and Credentials tab (per-project server URL, email, and API token). A global credentials section above handles the fallback credentials used by projects without overrides.

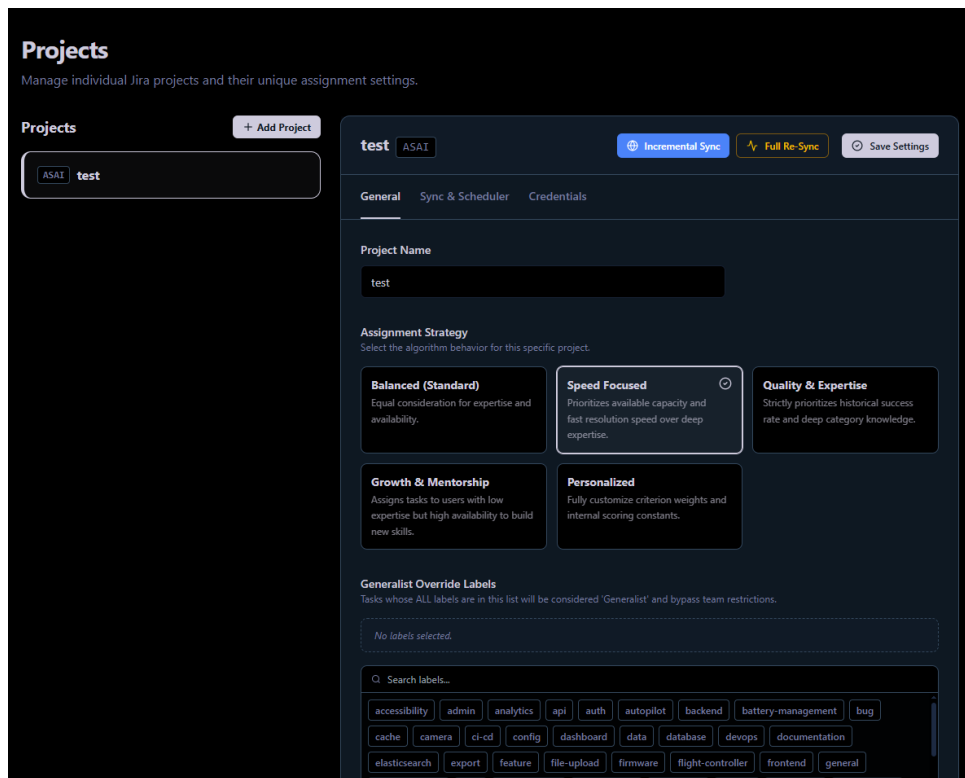


Figure 3: Projects Page

Weights Editor

Advanced UI for overriding the default scoring engine constants. Allows administrators to adjust the weight of each of the five criteria per project beyond the four built-in strategy presets. Also surfaces the dynamic priority score mapping fetched from Jira, which can be manually overridden for projects with non-standard priority names.

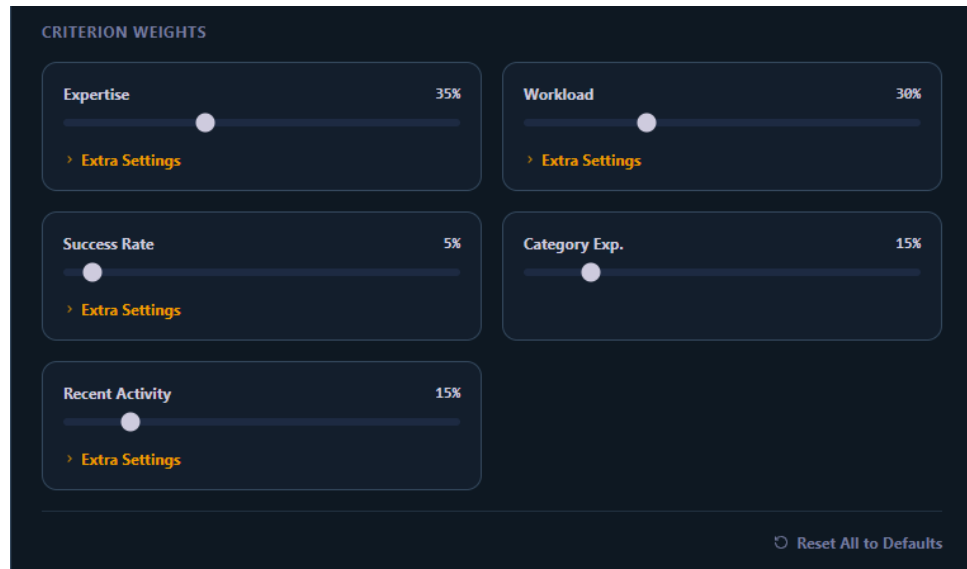


Figure 4: Weights Editor

Team Constraints

Interface for defining project-specific teams and routing rules. Users are searched from the Jira user directory and assigned to named teams. Team rules map Jira labels to teams, acting as hard constraints that filter candidates before soft scoring begins. Supports the GLOBAL team pattern for users who participate across all projects.

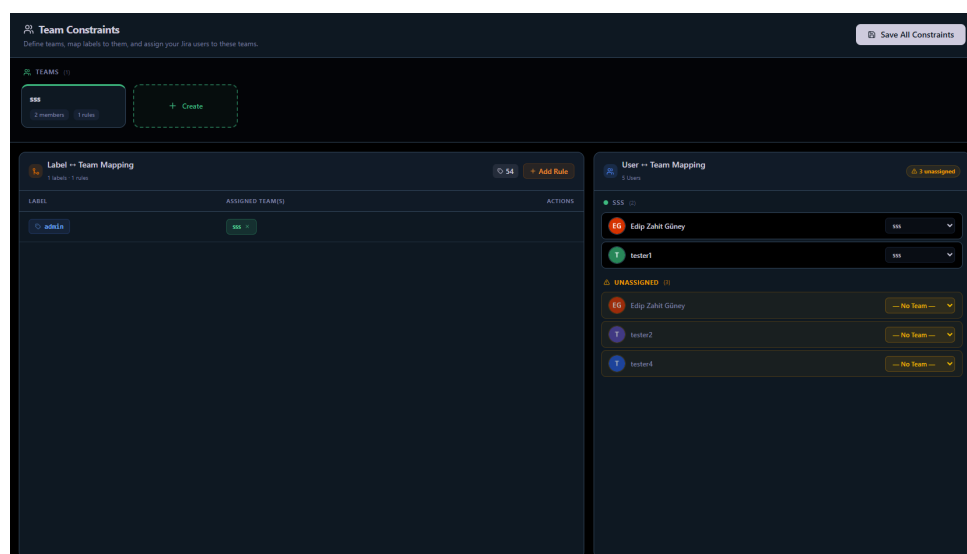
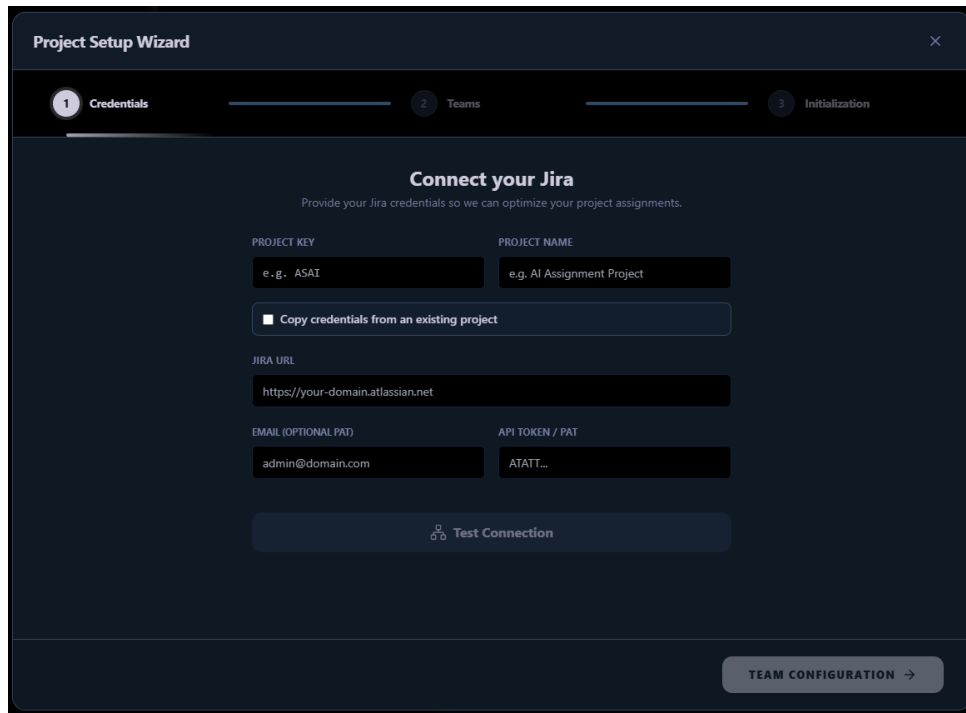


Figure 5: Team Constraints

Project Setup Wizard

A three-step modal overlay shown automatically when adding new projects. Guides the administrator through connecting to Jira, selecting a project from the live project list, defining teams, and completing the first sync before entering the dashboard. This wizard will appear whenever a new project is decided to be added.



The screenshot shows a 'Project Setup Wizard' modal with a dark theme. At the top, there's a title bar with a close button (X). Below it is a progress bar with three steps: 1. Credentials (active), 2. Teams, and 3. Initialization. The main content area is titled 'Connect your Jira' with a subtitle 'Provide your Jira credentials so we can optimize your project assignments.' It contains several input fields: 'PROJECT KEY' (with placeholder 'e.g. ASAI'), 'PROJECT NAME' (with placeholder 'e.g. AI Assignment Project'), a checkbox 'Copy credentials from an existing project', 'JIRA URL' (with placeholder 'https://your-domain.atlassian.net'), 'EMAIL (OPTIONAL PAT)' (with placeholder 'admin@domain.com'), and 'API TOKEN / PAT' (with placeholder 'ATATT...'). A 'Test Connection' button with a circular arrow icon is below these fields. At the bottom right, there is a 'TEAM CONFIGURATION →' button.

Figure 6: Project Setup Wizard

Project Context & Global State

`activeProject` is managed at the application root and injected via React context. Every API call includes the active project key as a request header, and all views subscribe to project changes so data always reflects the currently selected project. Deletion of the active project falls back to the first available project or redirects to the onboarding wizard if none remain.